

UNIVERSIDAD DE LAS FUERZAS ARMADAS – “ESPE-L”

ARQUITECTURA DE SOFTWARE

NRC: 31501



TEMA:

Informe Arquitectura Cloud

ESTUDIANTES:

Bustillos Oñate Jose Luis

Torres Alulema Josue David

FECHA: 21 de julio de 2026

ANALYTICORE

Problema de negocio

Actualmente, muchas empresas reciben comentarios, opiniones, reclamos y recomendaciones escritos por sus clientes. Estos textos contienen información importante para conocer el nivel de satisfacción de los usuarios, detectar problemas y tomar decisiones de mejora. Sin embargo, cuando la cantidad de mensajes aumenta, revisarlos uno por uno puede requerir demasiado tiempo y hacer que información relevante pase desapercibida.

Ante esta necesidad, la startup AnalytiCore busca ofrecer un servicio en línea que permita analizar textos de manera automática. El sistema debe identificar si el contenido expresa una opinión positiva, negativa o neutral y, al mismo tiempo, extraer las palabras más importantes del mensaje. De esta forma, se facilita la interpretación de la información sin depender únicamente de una revisión manual.

Solución propuesta y valor

La solución desarrollada consiste en una plataforma web en la que el usuario escribe un texto y solicita su análisis. La interfaz fue construida con React y presenta de manera sencilla el estado del proceso, el sentimiento detectado y las palabras clave obtenidas.

Cuando el usuario envía el texto, la información llega al servicio de submisión desarrollado con Python y FastAPI. Este servicio valida que el contenido sea correcto, crea un registro en PostgreSQL con el estado PENDIENTE y solicita al servicio Java que realice el análisis. El servicio Java cambia el estado a PROCESANDO, analiza las palabras del texto, determina el sentimiento, extrae las palabras clave y guarda los resultados. Finalmente, el registro cambia a COMPLETADO y la interfaz muestra la respuesta al usuario.

El valor principal de AnalytiCore es demostrar que una necesidad real puede resolverse mediante varios servicios especializados que trabajan en conjunto. Aunque el análisis implementado es sencillo, el prototipo sirve como base para incorporar en el futuro modelos de inteligencia artificial más avanzados, análisis de grandes cantidades de comentarios, generación de reportes y seguimiento histórico de resultados.

Beneficios de la arquitectura elegida

La arquitectura divide la aplicación en tres componentes principales: frontend, servicio Python y servicio Java. Esta separación facilita el mantenimiento,

porque cada componente tiene una responsabilidad definida. Por ejemplo, la interfaz puede modificarse sin cambiar completamente la lógica de análisis, mientras que el analizador Java puede mejorarse sin reconstruir toda la plataforma.

También se obtiene flexibilidad tecnológica, ya que cada lenguaje se utiliza en la tarea para la que resulta más conveniente. React permite construir una interfaz interactiva; Python facilita la validación y coordinación de solicitudes; y Java ofrece una estructura organizada para implementar el procesamiento del texto.

Otro beneficio es la escalabilidad. Si en el futuro aumenta la cantidad de usuarios, cada servicio podría recibir más recursos o ejecutarse en varias instancias de forma independiente. Además, los componentes se comunican mediante APIs REST, lo que evita una dependencia directa entre sus códigos internos.

Todos los servicios se encuentran empaquetados en contenedores Docker, permitiendo que la aplicación se ejecute de manera similar en diferentes equipos y reduciendo problemas de instalación. El estado de los trabajos se almacena en PostgreSQL y no dentro de la memoria de los servicios. Gracias a esto, la información se conserva aunque un componente se reinicie. En conjunto, la arquitectura propuesta ofrece una solución funcional, organizada, mantenible y preparada para futuras ampliaciones.